

Boxy

A friendly little programming language for kids... and it really works!

The Complete Language Manual

Version 1.0

The_X_Rider

<the_x_rider@proton.me>

Table of Contents

1. What is Boxy?
2. Installing and Running
3. Command Line Flags
4. Hello World
5. Comments
6. Boxes
7. Math
8. Strings
9. Constants
10. Comparisons
11. If, Otherwise
12. Loops
13. Functions
14. Lists
15. Memory: Kept vs Borrowed Boxes
16. Arrows (pointers)
17. Files
18. Helpers (threads)
19. Locks
20. Doors (networking)
21. Sleeping and Sizes
22. Asking the User
23. The Post-Mortem
24. Errors at a Glance
25. Limits
26. Examples and Lessons
27. Boxy in Real Languages
28. Author and License

1. What is Boxy?

Boxy is a small programming language for kids. It looks like simple English, and the metaphor at the heart of the language is a physical box: every value lives in a box with a name written on the side.

It is also a real language. Under the friendly vocabulary you will find variables, the stack and the heap, pointers, threads, mutexes, file handles, and TCP sockets. By the end of the twelve included lessons, a child can run a tiny chat program between two helpers over a real network door.

Errors are encouraging. Boxy never shouts. It starts every complaint with “Hmm,” “Oops,” or “Wait,” and tries to suggest a fix.

2. Installing and Running

Clone the repository and build the interpreter from source.

```
git clone https://github.com/thexrider/boxy.git
cd boxy
```

On Linux and macOS:

```
cc boxy.c -o boxy -pthread -lm
```

Boxy works on both — it points at the system C compiler. On Linux that is usually GCC; on macOS, Clang. Either is fine.

Run a program:

```
./boxy run examples/hello.bx
./boxy lesson 1
./boxy --help
```

3. Command Line Flags

Flag	What it does
run FILE	Execute the program in FILE.
lesson N	Run lesson N from the lessons/ folder.
--quiet	Skip the welcome line.
--strict	Exit non-zero if borrowed boxes were never returned, or files / sockets were left open.
--visual	When you say a box, draw it as a little ASCII box.
--help	Print the cheat sheet and exit.

Tip: use --strict in CI. A clean program ends with code 0; a program that leaked a borrowed box or left a socket open ends with a non-zero code.

4. Hello World

```
say "hello, world"
```

That is a complete program. Save it as `hello.bx` and run it:

```
./boxy run hello.bx
```

`say` prints a value. So does `show`. Both feel right.

5. Comments

```
# single line comment
```

```
###  
this whole block is a comment  
useful for headers and notes  
###
```

6. Boxes

A box is a place to keep a value. You give the box a name, put something into it, and ask for it again later.

There is one numeric type — every number is a 64-bit double internally. Boxy never says “int” to a child. There are also strings, lists, and the words `yes` / `no` for booleans.

The simplest form: assignment

```
name is "Ana"  
age is 7  
pi is 3.14
```

Or, more explicit, with the box-and-put form:

```
make a box called age  
put 9 in age  
say age
```

If you want to say it out loud that the box must come back, you *borrow* it instead. See chapter 15.

7. Math

Operations are spelled out in English.

Expression	Meaning
a plus b	$a + b$
a minus b	$a - b$
a times b	$a * b$

a divided by b	a / b
a mod b	a mod b (remainder)
a to b	a raised to the power of b
root of a	square root of a
random between a and b	a random integer in [a, b]

Examples:

```
x is 10 plus 5
y is 17 mod 5
p is 2 to 10
r is root of 144
d is random between 1 and 100
```

8. Strings

Strings are wrapped in double quotes. Concatenation is by listing values, separated by commas, on the same say line.

```
name is "world"
say "hello, ", name      # hello, world
```

Escape sequences

Escape	Meaning
\n	newline
\r	carriage return
\t	tab
\\	literal backslash
\"	literal double quote
\0	null byte

9. Constants

remember declares a value that cannot change. Trying to overwrite it is a friendly error.

```
remember PI as 3.14159
remember GREETING as "hi"
```

Use it for things that should never move underfoot — magic numbers, configuration, names you want to keep stable.

10. Comparisons

Operator	Example	Meaning
is	name is "Ana"	equals
is not	x is not 0	not equals
is at least	age is at least 18	$\geq$
is at most	score is at most 100	$\leq$
is greater than	x is greater than 0	$>$
is less than	x is less than 10	$<$
starts with	line starts with "PING "	string prefix
ends with	host ends with ".tv"	string suffix
contains	msg contains "switch"	substring

11. If, Otherwise

```
if age is at least 18 {
    say "grown up"
} otherwise {
    say "kid"
}
```

otherwise is the else branch.

12. Loops

Repeat N times

```
repeat 3 times { say "hi" }
```

Counted loop

```
count from 0 to 9 as i {
    say i
}
```

Both ends are inclusive. The loop variable (i here) is set on each round.

While

```
x is 0
while x is less than 10 {
    x is x plus 1
}
```

Early exit

Use stop to break out of any loop. Use skip to jump to the next round.

```
count from 1 to 100 as n {
```

```

    if n mod 2 is 0 { skip }
    if n is greater than 9 { stop }
    say n
}

```

13. Functions

Define with `teach`. Return a value with `give`. Call with `call NAME with ARGS`.

```

teach square with n {
    give n times n
}

```

```

say call square with 9          # 81

```

Multiple parameters are joined with `and`:

```

teach add with a and b {
    give a plus b
}

```

```

say call add with 3 and 4      # 7

```

Functions get their own scope — they cannot see or change the caller's variables. They can only return one value.

14. Lists

```

my friends are ["Ana", "Bea", "Caio"]

```

```

add "Diogo" to my friends      # append
take "Ana" from my friends     # remove first match
show my friends                # print all
how many my friends            # length

```

Lists hold strings or numbers. Mixing is allowed but rarely a good idea.

15. Memory: Kept vs Borrowed Boxes

This is the chapter that justifies the entire language. Boxy gives you two ways to make a box, and they behave like the stack and the heap in C — without ever using those words to a child.

Kept box: make

```

make a box called age
put 25 in age
say age

```

A kept box is cleaned up automatically when its program ends. You never free it. You cannot free it.

Borrowed box: borrow and return

```

borrow a box called score
put 100 in score
say score

```

```
return score          # give it back yourself
```

A borrowed box is on the heap. If you do not return it, the post-mortem at the end will tell you, by name and line number.

Rule of thumb: use `make a box` by default. Reach for `borrow` only when you need a value to outlive the function that created it. Same advice as in real C.

16. Arrows (pointers)

An arrow is a pointer to a box.

```
make a box called target
put 100 in target
```

```
make an arrow called ptr to target
say inside ptr          # 100 (dereference)
put 999 in ptr's box    # write through the arrow
say target              # 999
```

Dangling arrows

If you return a borrowed box while an arrow still points at it and then read through that arrow, Boxy stops with a clear error — by name, with the line number where the box was returned.

```
borrow a box called secret
put 42 in secret
```

```
make an arrow called peek to secret
return secret          # the box is gone
say inside peek        # ERROR: dangling arrow
```

This is the entire reason this chapter exists in the manual. In real C, that read would silently print garbage or crash. Here it gets caught.

17. Files

Writing

```
open file "/tmp/notes.txt" called f
write "hello\n" to f
close f
```

Reading

```
open file "/tmp/notes.txt" called f to read
read f save text
say text
close f
```

Modes

Suffix	Mode
(none)	write (truncate)

to read	read
---------	------

to append	append
-----------	--------

Forget `close` and the post-mortem will remind you. Same goes for sockets.

18. Helpers (threads)

`start helper` spawns a real pthread — literally another tiny you under the hood. `wait for` joins it.

```
start helper called worker {  
    say "I am the worker"  
}
```

```
wait for worker
```

Each helper gets its own copy of the parent's variables when it starts. Mutate the shared state through a lock (next chapter), or every helper is on its own.

19. Locks

`hold X` acquires a mutex named `X`. `let go X` releases it. The mutex is created automatically the first time you mention it.

```
counter is 0
```

```
start helper called worker {  
    hold counter  
    counter is counter plus 1  
    let go counter  
}
```

```
wait for worker  
say counter
```

Holding the same lock twice in one helper will deadlock. These are simple, non-recursive mutexes — because they always are.

20. Doors (networking)

Boxy wraps the BSD socket API in a metaphor: a door is a server socket, a knock is an incoming connection, and a guest is the connection you got from an `accept`.

What it really is	What you write
socket + bind + listen	open door called <code>srv</code> on port 9001
accept	wait for knock on <code>srv</code> save guest
connect	call door at "127.0.0.1" on port 9001 save line
send	send "hi" to guest

recv	read guest save msg
------	---------------------

close	close guest
-------	-------------

Echo server + client in one program

```

start helper called server {
    open door called srv on port 9001
    wait for knock on srv save guest
    read guest save msg
    send msg to guest
    close guest
    close srv
}

sleep 200 ms
call door at "127.0.0.1" on port 9001 save line
send "hello" to line
read line save reply
say reply
close line
wait for server

```

Heads up. TCP is a stream of bytes, not a stream of messages. One send can show up across two reads, or two sends can arrive in one read. That is true in every other language too. If you need messages, encode them yourself — newline-terminated, length-prefix, etc.

21. Sleeping and Sizes

Pause execution:

```

sleep 200           # 200 milliseconds (default)
sleep 250 ms
sleep 1 second
sleep 5 seconds

```

Ask Boxy how big a thing is, in bytes:

```

say size of box      # 8 (one number = one double)
say size of letter   # 1
say size of arrow    # 8 (a pointer)

```

22. Asking the User

```

ask "what is your name?" save name
say "hi, ", name

```

ask prints the prompt, reads one line from standard input, and stores the result in the box you named. If the input parses as a number, it is stored as a number; otherwise as a string.

23. The Post-Mortem

When the program ends, Boxy walks every box, file, and socket the program ever made, and reports the ones the child forgot to return or close — in the friendly tone, with line numbers.

Default exit code is 0 unless you passed `--strict`.

Example output:

```
--- Boxy: a friendly check-up ---
  You borrowed a box called 'score' on line 8 but never returned it.
  Borrowed boxes must come back!
  You opened the file 'log' on line 12 but never closed it.
  You left a connection called 'srv' open (from line 21).
```

24. Errors at a Glance

What you did	What you get
Used a name that does not exist	Hmm, I don't see a box... Did you mean NAME?
Read through an arrow whose box was returned	Wait — your arrow points at a box that was returned
Tried to put something in a box that was returned	Oops, that box was already returned
Wrote to a constant	You tried to change something you said to remember
Forgot to return a borrowed box	Post-mortem: borrowed box NAME is still out
Forgot to close a file or socket	Post-mortem: file / connection NAME still open
Divided (or modded) by zero	You divided by zero. That doesn't have an answer.
Read or wrote to a name that doesn't exist	Hmm, there is no box called NAME (with suggestion)

Every error includes the line number where it happened, the line itself, and most include the variable name involved.

25. Limits

Boxy is small on purpose. The interpreter has fixed-size buffers, modeled to be generous for kid-sized programs but obvious if you go wild.

Limit	Value
Maximum line length	1024 characters
Maximum identifier length	64 characters
Maximum string value length	1024 characters
Maximum kept + borrowed boxes	128
Maximum simultaneous helpers	32
Helper stack size	4 MB (set explicitly)
Open file handles	16

26. Examples and Lessons

Two folders. `examples/` are short programs you can run directly. `lessons/` are 12 friendly walk-throughs, one new idea each, run with `./boxy lesson N`.

Examples

File	Teaches
<code>examples/hello.bx</code>	the smallest program
<code>examples/name.bx</code>	ask + say
<code>examples/adder.bx</code>	functions
<code>examples/shopping.bx</code>	lists
<code>examples/diary.bx</code>	files
<code>examples/borrow.bx</code>	borrowed boxes
<code>examples/arrow.bx</code>	arrows / pointers
<code>examples/helpers.bx</code>	helpers and locks
<code>examples/echo.bx</code>	TCP echo server + client
<code>examples/chat.bx</code>	two helpers chatting over a socket

Lessons

Lesson	Teaches
1. hello	your first program
2. boxes	making and using a box
3. asking	reading the user's answer
4. math	plus, minus, times, divided by, ...
5. choosing	if / otherwise
6. loops	repeat, count from, while, stop, skip
7. functions	teach, give, call
8. lists	many things in a row

9. files	a tiny diary
10. borrowed boxes	the box you must return
11. helpers	two workers, one shared counter
12. doors	a tiny chat over a TCP socket

Run any of them with:

```
./boxy run examples/<name>.bx
./boxy lesson <N>
```

27. Boxy in Real Languages

Boxy is a real, working language, but the words are deliberately kid-friendly. Behind every Boxy idea is the same idea you would meet in C, Python, Go, Java, or JavaScript. This chapter exists so a curious kid (and the parent reading along) knows what is actually being learned, and where it leads next.

Word-by-word translation

Boxy	C	Python / others
make a box called age	int age;	age = 0
put 7 in age	age = 7;	age = 7
age is 7	int age = 7;	age = 7
borrow a box called score	int *score = malloc(...);	(no direct equivalent)
return score	free(score);	(garbage collector does it)
make an arrow called ptr to age	int *ptr = &age;	(implicit; objects are references)
inside ptr	*ptr	(implicit dereference)
put 999 in ptr's box	*ptr = 999;	(implicit)
if X { ... } otherwise { ... }	if (X) { ... } else { ... }	if X: ... else: ...
repeat N times { ... }	for (i=0; i<N; i++) {...}	for _ in range(N): ...
count from A to B as i { ... }	for (i=A; i<=B; i++) {...}	for i in range(A, B+1): ...
while X { ... }	while (X) { ... }	while X: ...
teach square with n { give n times	int square(int n){return n*n;}	def square(n): return n*n
call square with 5	square(5)	square(5)
start helper called w { ... }	pthread_create(&t, ...);	threading.Thread(target=...).start()
wait for w	pthread_join(t, NULL);	t.join()

hold counter / let go counter	<code>pthread_mutex_lock/unlock</code>	<code>lock.acquire() / lock.release()</code>
open file "x" called f	<code>FILE *f = fopen("x", "w");</code>	<code>f = open("x", "w")</code>
close f	<code>fclose(f);</code>	<code>f.close()</code>
open door called srv on port N	<code>socket()+bind()+listen()</code>	<code>socket.socket(); s.bind(); s.listen()</code>
wait for knock on srv save g	<code>accept(srv, ...);</code>	<code>g, _ = s.accept()</code>
call door at "H" on port N	<code>socket()+connect()</code>	<code>s = socket.create_connection(...)</code>
send X to g / read g save msg	<code>send(fd, ...); recv(fd, ...);</code>	<code>s.send(...); s.recv(...)</code>

The big ideas

Variables. A box with a name and a value. Every language has this. Boxy says make a box; Python and JavaScript skip the declaration; C, Java and Go make you say the type.

Stack vs heap. A kept box lives on the stack — the computer automatically gives the memory back when the function or program ends. A borrowed box lives on the heap — you asked for it, you give it back. In C, that's `malloc` and `free`. In Python, JavaScript, Go, Java the language hides this from you (a garbage collector does it later). Boxy makes the difference visible on purpose.

Pointers. An arrow that says "the value you want is over there." In C and C++ they are everywhere and easy to break. In Python and JavaScript every variable that holds an object is secretly an arrow — you just never see the word. Boxy makes the arrow visible, and refuses to let you read through one whose box was returned (a "dangling pointer").

Functions and scope. A small machine you teach once and use many times. Boxy's teach is Python's `def`, JavaScript's `function`, C's function definition. `give` is `return`.

Threads and locks. A helper is a real OS thread (`pthread` under the hood). The lock is a real mutex. The same shape of code works in C with `pthread_create` and `pthread_mutex_lock`, in Python with `threading.Thread` and `threading.Lock`, in Go with `go` and `sync.Mutex`.

Files and sockets. Boxy's doors and guests are TCP server and client sockets. The exact same operations exist in every other language: `bind`, `listen`, `accept`, `connect`, `send`, `recv`. The metaphor only changes the names.

Where to go next

If a kid likes Boxy, the natural next steps are:

Python — closest in feel. `say` becomes `print`, `ask` becomes `input`, lists work the same. No borrowing — Python cleans up by itself.

C — to see what borrowing really costs. Everything Boxy warns about (forgetting `return`, dangling arrows, unclosed files) becomes silent crashes in C, and finding them is a whole career.

Go — for helpers and doors at scale. Goroutines look like Boxy helpers. Channels are a different way to make them talk.

JavaScript — for things people see in a browser. Variables and functions feel familiar. Threads work very differently.

Whatever language comes next, the ideas Boxy taught are the ones that travel. Pointers, threads, sockets, the heap, the stack — these words show up in job interviews and in real programs. Boxy is just the first place a kid meets them, with all the sharp corners rounded off.

28. Author and License

The_X_Rider <the_x_rider@proton.me>

Boxy is licensed under **GPL-3.0-or-later**. You are free to use, study, share, and modify it. If you distribute a modified version, that version must also stay free under the same license. Boxy will always be free software.

End of manual. Now go make a box.